C++17 `std::string_view`: Stop Copying Strings Like It's 2003

How to achieve safer memory management and significant speedups with one simple type change!

Sagar Following 12 min read · Mar 24, 2026

`std::string_view` is a non-owning, read-only window into a string. It doesn't allocate memory, it doesn't copy data, and it makes your string-heavy code measurably faster. It also has a few sharp edges that will bite you if you're not paying attention. Lets dig in.

. . .

The Problem With `std::string` and Why We Should Care ?

Let's set the scene. You're writing a function that takes a string and does some read-only work on it — maybe parses it, searches through it, logs it, whatever. You write:

```
void processLog(const std::string& log) {
    // just reads from it, never modifies it
}
```

Looks fine, right? Now you call it with a string literal:

```
processLog("ERROR: disk full");         // implicit temp string
processLog(someCharPointer);            // same deal
processLog(std::string("user: bob"));   // explicit temp, still allocates
```

Here's the thing. When you pass a `const char*` or a string literal to a `const std::string&`, C++ helpfully constructs a temporary `std::string` for you. That means a **heap allocation**. Every. Single. Time.

And what about `std::string::substr` ?
Also an allocation. Always returns a new `std::string`. Always copies the data.

```
std::string big = "the quick brown fox jumps over the lazy dog";
std::string part = big.substr(4, 5);  // "quick" — heap allocated, data copied
```

This is fine for one-off operations. But if you're doing this in a tight loop, in a parser, in a server handling thousands of requests per second — you're leaving a lot of performance on the table.

. . .

Enter C++17 `std::string_view`

C++17 introduced `std::string_view` (in `<string_view>`) to solve exactly this.

The idea is dead simple: **instead of owning and copying string data, just remember where it is and how long it is.**

That's it. Two fields. A pointer and a length. No heap. No copies.

```
#include <string_view>

void processLog(std::string_view log) {
    // reads from it, zero copies, zero allocations
}
```

Now those same calls from before ? Not a single heap allocation happens for `string_view` construction.

How It Actually Works (Memory Model) ?

Here's what's going on under the hood. A `std::string_view` is basically this:

```
struct string_view {
    const char* ptr;   // pointer to the start of the string data
    size_t      len;   // how many characters to consider
};
```

When you create a `string_view`, you're not creating a string. You're creating a *lens* pointed at an existing string.

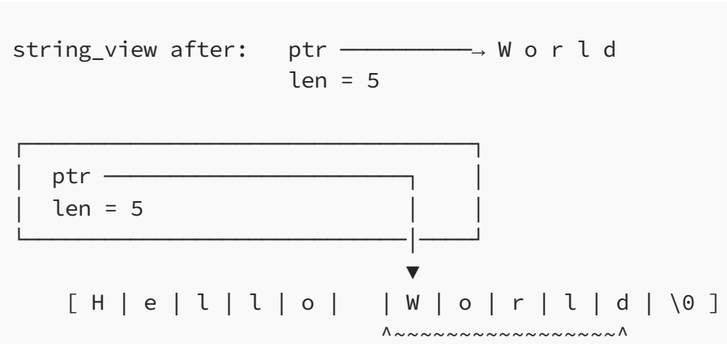
```
Original string (std::string):
┌──────────────────────────────────────────────────────┐
│  heap-allocated buffer                               │
│  [ H | e | l | l | o |   | W | o | r | l | d | \0 ]   │
│  ^                                                   │
│  ptr                     size=11    capacity=15      │
└──────────────────────────────────────────────────────┘

std::string_view pointing into the same buffer:
┌──────────────────────────────┐
│  ptr ──────────────────┐     │
│  len = 11              │     │
└────────────────────────┼─────┘
                         ▼
                        [ H | e | l | l | o |   | W | o | r | l | d | \0 ]
                        ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^
                        This is what string_view "sees"
                        No copy. No allocation.
```

Now here's the cool part — `remove_prefix` and `remove_suffix`:

```
After sv.remove_prefix(6):

string_view before:  ptr → H e l l o   W o r l d
                     len = 11
```



This is **O(1)**. Always. No matter how large the string is.

Construction: What It Accepts ?

One of the nicest things about `string_view` is how flexible it is:

```cpp
#include <string>
#include <string_view>

// From a string literal — no allocation
std::string_view sv1 = "hello world";

// From a std::string — no allocation, just borrows the buffer
std::string s = "hello world";
std::string_view sv2 = s;

// From a const char* and explicit length
const char* buf = "hello world extra stuff";
std::string_view sv3(buf, 11);  // "hello world"

// From a C-array
char arr[] = "hello";
std::string_view sv4(arr, sizeof(arr) - 1);  // exclude null terminator

// C++17 string literal suffix (sv)
using namespace std::string_view_literals;
auto sv5 = "hello"sv;  // type is std::string_view, not const char*
```

What it does **not** accept (at least, not without you noticing):

```
// This works, but now you have a lifetime problem (more on that later)
std::string_view danger = std::string("temporary");  // dangling!
```

What You Can Do With It ?

`std::string_view` supports almost everything you'd want to do with a read-only string:

```
std::string_view sv = "the quick brown fox";

// Basics
sv.size();          // 19
sv.length();        // same thing
sv.empty();         // false
sv[0];              // 't'
sv.front();         // 't'
sv.back();          // 'x'
sv.data();          // const char* pointer to the start

// Slicing
sv.substr(4, 5);      // "quick" — O(1), no allocation!
sv.remove_prefix(4);  // sv is now "quick brown fox"
sv.remove_suffix(4);  // sv is now "quick brown"

// Searching
sv.find("brown");       // returns position or string_view::npos
sv.rfind("o");          // last occurrence
sv.find_first_of("aeiou");   // first vowel
sv.find_last_not_of(" ");    // strip trailing space logic

// Comparison — compares by content
sv == "the quick brown fox";  // true
sv.starts_with("the");        // true  (C++20, but worth mentioning)
sv.ends_with("fox");          // true  (C++20)

// Convert back to std::string when you need ownership
std::string owned(sv);        // yes, this allocates
std::string owned2 = std::string(sv);  // same thing
```

What you **cannot** do:

```
sv[0] = 'T';           // compile error — it's read-only
sv += " jumps";        // compile error — no mutation
sv.push_back('!');     // doesn't exist
sv.c_str();            // doesn't exist — not null-terminated!
```

> ***That last one is important:***
>
> `std::string_view` ***is not guaranteed to be null-terminated.***
>
> *It's just a pointer and a length.*
>
> *Some APIs that take* `const char*` *and rely on the null terminator will break.*
>
> *You've been warned.*

. . .

Performance Benchmarks That'll Make You Sweat

Let's get into the numbers. Here are several scenarios comparing `std::string` vs `std::string_view`.

Benchmark 1: Substring creation in a loop

This is the bread-and-butter use case. We take a large string and create 10 million substrings of fixed length from random positions.

```
// Scenario: 10,000,000 substrings of length 30 from a ~500KB text file
```

```
// std::string version:
for (int i = 0; i < 10'000'000; ++i) {
    std::string sub = bigString.substr(randPos[i], 30);  // heap alloc every tim
    (void)sub;
}

// std::string_view version:
std::string_view sv = bigString;
for (int i = 0; i < 10'000'000; ++i) {
    std::string_view sub = sv.substr(randPos[i], 30);    // zero allocation
    (void)sub;
}
```

Results (GCC, -O2):

```
┌──────────────────────────────────────────────────────────┐
│  Benchmark: 10M substring operations on ~500KB string    │
├──────────────────────────────┬──────────────┬────────────┤
│  Method                      │  Time (s)    │  Relative  │
├──────────────────────────────┼──────────────┼────────────┤
│  std::string::substr         │  0.816 s     │  45x slower│
│  std::string_view::substr    │  0.018 s     │  1x (baseline)│
└──────────────────────────────┴──────────────┴────────────┘
```

Benchmark 2: Complexity vs. Substring Size

This one shows *why* the gap exists.

`std::string::substr` is O(n) — it copies n characters.

`std::string_view::substr` is O(1) — it just adjusts a pointer and a number.

```
┌──────────────────────────────────────────────────────────────┐
│  Substr time vs. substring length (10M operations, -O2)      │
├──────────────┬──────────────────────────┬────────────────────┤
│  Length      │  std::string (ms)        │  std::string_view (ms)  │
├──────────────┼──────────────────────────┼────────────────────┤
│  10 chars    │  710                     │  18                │
│  30 chars    │  816                     │  18                │
│  100 chars   │  956                     │  18                │
│  1000 chars  │  2240                    │  18                │
```

```
| 10000 chars  | 17800                | 18                          |
└──────────────┴──────────────────────┴─────────────────────────────┘
```

> *The string_view time is **flat**. It doesn't care if the substring is 10 characters or 10,000 characters. The string version, meanwhile, is suffering more and more as length grows. This is exactly what O(1) vs O(n) looks like in practice.*

Benchmark 3: Function call overhead

Passing strings into functions that just read them:

```
void doSomething(const std::string& s) { /* read-only work */ }
void doSomethingView(std::string_view sv) { /* read-only work */ }

// Call with a string literal:
doSomething("a string literal");       // temp std::string constructed → heap al
doSomethingView("a string literal");   // no allocation

// Call with a char*:
const char* cstr = "a string literal";
doSomething(cstr);       // heap alloc (unless small string optimization kicks i
doSomethingView(cstr);   // no allocation
```

```
┌──────────────────────────────────────────────────────────────────┐
│  10M function calls with const char* argument                    │
├──────────────────────────────┬───────────────────────────────────┤
│  const std::string& param    │  ~620 ms  (allocates temps)       │
│  std::string_view param      │  ~18 ms   (zero allocations)      │
└──────────────────────────────┴───────────────────────────────────┘
```

A note on Small String Optimization (SSO):

Before you cry "but my strings are tiny!", be aware that `std::string` does have a trick up its sleeve: **Small String Optimization**. Strings below ~15 chars (GCC/MSVC) or ~23 chars (Clang) are stored directly inside the string object itself, skipping the heap allocation.

```
SSO threshold (approximate):
┌──────────────────────────┬──────────────────────────┐
│  Compiler                │  SSO threshold           │
├──────────────────────────┼──────────────────────────┤
│  GCC                     │  ≤ 15 chars              │
│  MSVC                    │  ≤ 15 chars              │
│  Clang (libc++)          │  ≤ 22 chars              │
└──────────────────────────┴──────────────────────────┘
```

So for short strings, `std::string` isn't as bad as it looks. But `string_view` still wins for `substr` operations and is never *worse*.

. . .

Real-World Scenarios Where It Shines

Scenario A: Writing a parser

Parsers eat and slice strings constantly. Before `string_view`, every token you extracted was a heap allocation:

```cpp
// Old way: every token is a heap allocation
std::vector<std::string> tokenize(const std::string& input) {
    std::vector<std::string> tokens;
    size_t start = 0, end;
    while ((end = input.find(' ', start)) != std::string::npos) {
        tokens.push_back(input.substr(start, end - start));  // alloc!
        start = end + 1;
    }
    tokens.push_back(input.substr(start));  // alloc!
    return tokens;
}
```

```cpp
// New way: no allocations during tokenization
std::vector<std::string_view> tokenize(std::string_view input) {
    std::vector<std::string_view> tokens;
    size_t start = 0, end;
```

```
    while ((end = input.find(' ', start)) != std::string_view::npos) {
        tokens.push_back(input.substr(start, end - start));  // just pointer mat
        start = end + 1;
    }
    tokens.push_back(input.substr(start));  // just pointer math
    return tokens;
}
```

Memory diagram for the parser scenario:

```
Input string (owned by caller):
"the quick brown fox"
 ^   ^     ^     ^
 |   |     |     |
 |   |     |     └── token[3]: string_view { ptr+10, len=3 }
 |   |     └──────── token[2]: string_view { ptr+4,  len=5 }
 |   └────────────── token[1]: string_view { ptr+0,  len=3 }
 └──────────────────── token[0]: string_view { ptr+0,  len=3 }

All tokens point INTO the original string. Zero copies.
```

Scenario B: Configuration file reading

You've read a config file into a single `std::string`. Now you're pulling out key-value pairs. With `string_view`, none of those lookups allocate:

```
std::string fileContents = readFile("config.ini");  // one allocation
std::string_view sv = fileContents;

// All of this is zero-allocation:
auto line = sv.substr(0, sv.find('\n'));
auto key  = line.substr(0, line.find('='));
auto val  = line.substr(line.find('=') + 1);
```

Scenario C: Accepting multiple string types in an API

Before `string_view`, if you wanted your function to accept both `std::string` and `const char*` efficiently, you needed overloads:

```
// Old way: two overloads or a conversion cost
void log(const std::string& msg);
void log(const char* msg);
```

With `string_view`, one function accepts everything:

```
// New way: one function, accepts std::string, const char*, char[],
//          string literals, string_view — all without copying
void log(std::string_view msg);
```

. . .

The Danger Zone: Things That Will Ruin Your Day

Alright, fun time is over. Let's talk about the ways `string_view` will stab you in the back if you're not careful.

Danger #1: Dangling References (The Classic)

`string_view` doesn't own the data. If the thing it's pointing to disappears, your `string_view` is pointing at garbage. This is undefined behavior and the compiler may not warn you.

```
std::string_view bad() {
    std::string local = "hello world, i will disappear";
    return local;  // string_view returned, local destroyed → dangling pointer!
}
```

```
// At the call site:
auto sv = bad();  // sv.data() points to freed memory
std::cout << sv;  // undefined behavior — crash, garbage, or "works" by accident
```

The golden rule: A `string_view` must never outlive the string it's viewing.

```
Lifetime diagram:

std::string s = "hello";   ← string alive
│
├── sv = std::string_view(s);  ← sv created, safe here
│         │
│         └── using sv here is fine
│
s goes out of scope  ← string destroyed
│
└── using sv here is UNDEFINED BEHAVIOR
```

Danger #2: Temporary Strings

This is a sneaky version of the previous problem:

```
// This LOOKS fine but isn't
std::string_view sv = std::string("hello");
// The temporary std::string is destroyed at end of this statement!
// sv is already dangling.

// Also bad:
std::string_view sv2 = std::string("prefix") + std::string("suffix");
// Same problem. Concatenation creates a temporary, it's immediately destroyed.
```

GCC and Clang will sometimes warn about this with `-Wdangling`, but don't bet on it catching everything.

Danger #3: Not Null-Terminated

`std::string_view` is **not** guaranteed to be null-terminated. This breaks any API expecting a C-style `const char*`:

```
std::string_view sv = "hello world";
sv = sv.substr(0, 5);  // sv = "hello", but NOT null-terminated

// These will break or give wrong results:
printf("%s", sv.data());   // reads past "hello" until random \0
std::ofstream f(sv.data()); // might open the wrong file name

// The right way if you need null-termination:
std::string safe(sv);      // copy to string first
printf("%s", safe.c_str()); // now it's null-terminated
```

```
What sv.data() actually points at after substr:

Original buffer:  h e l l o   w o r l d \0
                  ^~~~~~~~~~~~^
sv = substr(0,5)  ptr points here, len=5

sv.data() returns ptr, but there's no \0 at position 5.
Any C function will happily read "hello world" instead of "hello".
```

Danger #4: Storing `string_view` in Long-Lived Structures

This is where people get burned in real codebases:

```
struct Config {
    std::string_view name;   // who owns this?
    std::string_view value;
};

Config parseConfig(const std::string& line) {
    // line is a local copy passed by value
    // string_view of it will dangle after function returns!
    auto eq = line.find('=');
    return { line.substr(0, eq), line.substr(eq + 1) };
```

```
    //        ^ dangling!             ^ dangling!
}
```

If your struct needs to **own** the data for any significant lifetime, use `std::string`. If you're just passing `string_view` through a call stack that's all within a single expression or short-lived scope, you're fine.

Danger #5: Comparing With `std::string` in Containers

```
std::unordered_map<std::string, int> myMap;
myMap["hello"] = 42;

std::string_view sv = "hello";

// This might not compile or might be slow depending on your STL version:
auto it = myMap.find(sv);  // may construct a temporary std::string

// Better: use a heterogeneous lookup map (C++20) or convert explicitly
```

This is less “crashes” and more “defeats the purpose.” You just wanted to avoid the copy, and now you’re making one anyway.

. . .

Quick Decision Guide: `string` vs `string_view`

```
Do you need to read from a string without modifying it?
|
|   YES
|    |
|    |   or outlive the caller's string)?
|    |    |
|    |    ├── YES → Use std::string (or std::string&&)
```

```
|    |    |
|    |    └── NO  → Use std::string_view
|    |
|    └── Will you call APIs that need null-termination (printf, fopen, etc.)?
|         |
|         ├── YES → Use const std::string& (or convert inside the function)
|         |
|         └── NO  → Use std::string_view
|
└── NO (you need to modify the string)
     |
     └── Use std::string& or std::string (depending on ownership needs)
```

Another useful heuristic:

```
Is your function a...

  "Sink" (takes ownership)?   →  std::string (by value or &&)
  "Observer" (reads only)?    →  std::string_view
  "Modifier" (mutates)?       →  std::string&
  "Producer" (creates new)?   →  returns std::string
```

Summary Cheat Sheet:

```
┌──────────────────────────────────────────────────────────────┐
│                  std::string_view Quick Reference            │
├──────────────────────────┬───────────────────────────────────┤
│  Header                  │  <string_view>                    │
│  Available since         │  C++17                            │
│  Owns data?              │  No                               │
│  Heap allocates?         │  Never                            │
│  Null-terminated?        │  Not guaranteed                   │
│  Mutable?                │  No                               │
│  substr complexity       │  O(1)                             │
│  std::string substr      │  O(n)                             │
├──────────────────────────┼───────────────────────────────────┤
│  DO use for              │  Read-only string params          │
│                          │  Parsing and slicing operations   │
│                          │  Replacing const std::string& parameters  │
│                          │  Short-lived views within a scope │
```

```
┌──────────────────────────┬──────────────────────────────────────────┐
│  DON'T use for           │  Storing long-term (dangle risk)         │
│                          │  APIs needing null-termination           │
│                          │  Struct members (usually)                │
│                          │  Modifying strings                       │
│                          │  Returning views to local strings        │
├──────────────────────────┼──────────────────────────────────────────┤
│  Common gotchas          │  Dangling from temporaries               │
│                          │  Non-null-terminated data() calls        │
│                          │  Lifetime assumptions in structs         │
│                          │  Heterogeneous container lookup          │
└──────────────────────────┴──────────────────────────────────────────┘
```

. . .

One Last Code Pattern to Bookmark

The “pass and don’t copy” pattern — use this basically everywhere you’d have written `const std::string&` before:

```cpp
// Before C++17 — the "safe but slow" idiom
void process(const std::string& input) {
    auto part = input.substr(5, 10);  // heap allocation
    if (input.find("error") != std::string::npos) {
        // ...
    }
}

// C++17 — the "safe AND fast" idiom
void process(std::string_view input) {
    auto part = input.substr(5, 10);  // O(1), no allocation
    if (input.find("error") != std::string_view::npos) {
        // ...
    }
}
```

That’s it. Go forth and stop copying strings unnecessarily. Your profiler will thank you.

. . .

Closing Thoughts

And look — if you've made it this far, you now know more about `std::string_view` than roughly 60% of C++ developers who are still blissfully passing `const std::string&` everywhere and wondering why their parser is slow. You're going to use `string_view` correctly, avoid the dangling pointer trap, remember it's not null-terminated, and feel quietly smug about it at code review. Just don't be that person who refactors *everything* to `string_view` in one giant PR, breaks three services because someone stored a view of a temporary, and then blames this tutorial. You were warned. Twice. In its own section. With ASCII art.

. . .

Compiled with references from Modernes C++, cppreference, and the accumulated suffering of people who learned this the hard way.

Programming | Software Development | C Plus Plus Language | Cpp17 | Coding